

Each question below is based on the textbook's definition of the **ListNode** structure (given below):

```
struct ListNode {
    double value ;
    ListNode *next ;
    // Constructor
    ListNode (double new_value, ListNode *new_next = nullptr) {
        value = new_value ;
        next = new_next ;
    }
} ;
```

10. Write a function with the heading:

```
void printSecond(ListNode *ptr)
```

which prints the value stored in the *second* node of the list passed as a parameter. The function should print an error message and stop running the program if the list passed to it has less than 2 nodes.

```
void printSecond(ListNode *ptr) {
    // Check if the list is empty or has only one node
    if (ptr == nullptr || ptr->next == nullptr) {
        cerr << "Error: The list has less than 2 nodes." << endl;
        exit(EXIT_FAILURE); // Stops running the program
    }

    // Print the value stored in the second node
    cout << ptr->next->value << endl;
}
```

12. Write a function with the heading:

```
ListNode * removeFirst(ListNode *ptr)
```

which is passed a linked list as parameter and returns the *new* head of the list: That is, it removes the first node and returns what is left. The function should deallocate the storage of the removed node. The function returns **nullptr** if the list passed to it is empty.

```

ListNode * removeFirst(ListNode *ptr) {
    // If the list is empty, return nullptr
    if (ptr == nullptr) {
        return nullptr;
    }

    // Keep track of the second node it is the new head
    ListNode *newHead = ptr->next;

    // Deallocate
    delete ptr;

    // Return the rest of the list
    return newHead;
}

```

For each of the following program fragments, predict what the output will be.

14.

```

ListNode *p = new ListNode(56.4) ;
p = new ListNode(34.2, p) ;
cout << (*p).value << endl << p->value ;

```

Output:

34.2

34.2

16.

```

ListNode *p = new ListNode(56.4, new ListNode(31.5)) ;
ListNode *q = p ;
while (q->next != nullptr)
    q = q->next ;
cout << q->value ;

```

Output

31.5